Lecture 3.1 – Feedback and Proportional Control

Week 3: Making the Motor Respond to Speed Error

Learning Objectives

By the end of this lecture, you should be able to:

- explain why feedback is useful for motor-speed control;
- calculate the speed error $e(t) = r(t) - y(t)$;
- explain the proportional-control law $u(t) = K_p e(t)$;
- identify the units of K_p;
- derive the closed-loop transfer function for proportional control;
- explain how K_p changes steady-state error and response speed; and
- recognize PWM saturation as an important practical limit.

1. Why Open-Loop Control Is Not Enough

In Week 1, we used a fixed PWM value to command the motor.
For example,

$$u(t) = 160.$$

If this produces the desired speed under one condition, it may not produce the same speed when:

- a load is added;
- battery or supply voltage changes;
- friction changes;
- the motor warms up.

The problem is simple:

> An open-loop controller does not know whether the actual motor speed is correct.

Suppose the desired speed is

$$r = 150 \text{ RPM},$$

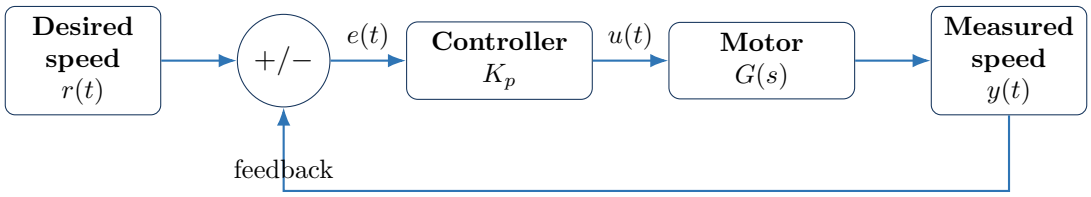
but the motor is actually running at

$$y = 120 \text{ RPM}.$$

A fixed PWM command does not automatically correct the difference.

2. Feedback Uses the Measured Speed

With feedback, the encoder measures motor speed and sends that information back to the controller.



The controller compares desired speed with measured speed.

$$\boxed{e(t) = r(t) - y(t)}$$

where

- $r(t)$ = reference or desired speed;
- $y(t)$ = measured speed;
- $e(t)$ = speed error.

3. Interpreting the Error

Suppose

$$r = 150 \text{ RPM}.$$

Measured speed y	Error $e = r - y$	Meaning
100 RPM	+50 RPM	motor is too slow
145 RPM	+5 RPM	motor is slightly slow
150 RPM	0 RPM	motor is at target
165 RPM	−15 RPM	motor is too fast

The sign and size of the error tell the controller what correction is needed.

4. Proportional Control

The simplest feedback controller is a proportional controller.

$$\boxed{u(t) = K_p e(t)}$$

The controller output is proportional to the current error.
If the error is large, the controller makes a large correction.
If the error becomes small, the correction becomes smaller.

Units of Kp

For our motor,

$$u(t) = \text{PWM command}$$

and

$$e(t) = \text{RPM}.$$

Therefore,

$$\boxed{K_p \text{ has units of } \frac{\text{PWM}}{\text{RPM}}.}$$

5. Simple Numerical Example

Suppose

$$r = 150 \text{ RPM}$$

and

$$K_p = 2 \ \frac{\text{PWM}}{\text{RPM}}.$$

If the measured speed is

$$y = 100 \text{ RPM},$$

then

$$e = r - y = 150 - 100 = 50 \text{ RPM}.$$

The proportional-control command is

$$u = K_p e = 2(50) = 100.$$

Thus,

$$\boxed{u = 100 \text{ PWM}.}$$

Later, if the speed rises to

$$y = 140 \text{ RPM},$$

then

$$e = 10 \text{ RPM}$$

and

$$u = 2(10) = 20.$$

Proportional control automatically reduces the correction as the error becomes smaller.

6. The Complete Feedback Loop

The control calculation repeats continuously:

1. measure motor speed;
2. calculate
$$e = r - y;$$
3. calculate
$$u = K_p e;$$

4. limit the command to the available PWM range;
5. send PWM to the motor driver;
6. repeat.

In code, the basic idea is:

```
error = desiredRPM - measuredRPM;

pwm = Kp * error;

pwm = constrain(pwm, 0, 255);

analogWrite(ENA, pwm);
```

7. Closed-Loop Transfer Function

The plant model from Week 2 is

$$G(s) = \frac{Y(s)}{U(s)}.$$

For proportional control,

$$U(s) = K_p E(s).$$

The feedback error is

$$E(s) = R(s) - Y(s).$$

Therefore,

$$Y(s) = G(s)K_p[R(s) - Y(s)].$$

Expand:

$$Y(s) = K_p G(s)R(s) - K_p G(s)Y(s).$$

Move the $Y(s)$ terms together:

$$Y(s)[1 + K_p G(s)] = K_p G(s)R(s).$$

Therefore,

$$\boxed{\frac{Y(s)}{R(s)} = \frac{K_p G(s)}{1 + K_p G(s)}.}$$

This is the standard unity-feedback closed-loop transfer function.

8. Apply the First-Order Motor Model

Our motor model is

$$G(s) = \frac{K}{\tau s + 1}.$$

Substitute into the closed-loop formula:

$$\frac{Y(s)}{R(s)} = \frac{K_p \dfrac{K}{\tau s + 1}}{1 + K_p \dfrac{K}{\tau s + 1}}.$$

Simplifying gives

$$\boxed{\frac{Y(s)}{R(s)} = \frac{K K_p}{\tau s + 1 + K K_p}.}$$

We can also write this in standard first-order form:

$$\frac{Y(s)}{R(s)} = \frac{\dfrac{K K_p}{1 + K K_p}}{\dfrac{\tau}{1 + K K_p} s + 1}.$$

This tells us two important things immediately.

9. Effect of Kp on Steady-State Error

The closed-loop steady-state gain is

$$\frac{K K_p}{1 + K K_p}.$$

For a constant reference r,

$$y_{ss} = \frac{KK_p}{1 + KK_p} r.$$

Therefore,

$$e_{ss} = r - y_{ss}.$$

Substituting gives

$$\boxed{e_{ss} = \frac{r}{1 + KK_p}.}$$

As K_p increases,

$$e_{ss}$$

decreases.

However, for finite K_p,

$$e_{ss} \neq 0.$$

A proportional controller usually reduces steady-state error, but it does not completely eliminate it. This is one reason we will add integral control in Week 4.

10. Effect of Kp on Response Speed

The closed-loop time constant is

$$\boxed{\tau_{cl} = \frac{\tau}{1 + KK_p}.}$$

As K_p increases,

$$\tau_{cl}$$

decreases.

Therefore, the ideal closed-loop response becomes faster.

Increasing K_p has two useful ideal effects:

- smaller steady-state error;
- faster response.

But real hardware places limits on how large K_p can be.

11. Worked Example: Closed-Loop Motor

Suppose the Week 2 motor model is

$$G(s) = \frac{0.775}{0.55s + 1}.$$

Choose

$$K_p = 5\ \frac{\text{PWM}}{\text{RPM}}.$$

Then

$$KK_p = 0.775(5) = 3.875.$$

The closed-loop transfer function is

$$\frac{Y(s)}{R(s)} = \frac{3.875}{0.55s + 4.875}.$$

In standard first-order form,

$$\boxed{\frac{Y(s)}{R(s)} = \frac{0.795}{0.113s + 1}.}$$

Therefore,

$$\tau_{cl} = \frac{0.55}{4.875} \approx 0.113 \text{ s}.$$

For a reference of

$$r = 150 \text{ RPM},$$

the ideal steady-state speed is

$$y_{ss} = 0.795(150) \approx 119.2 \text{ RPM}.$$

The steady-state error is

$$e_{ss} = 150 - 119.2 = 30.8 \text{ RPM}.$$

12. Why Does P Control Leave an Error?

At steady state, the motor still needs a nonzero PWM command to keep rotating.
But with pure proportional control,

$$u = K_p e.$$

If the error became exactly zero,

$$e = 0,$$

then

$$u = 0.$$

The controller would no longer provide the PWM needed to maintain the speed.
Therefore, the system settles with some nonzero error so that

$$K_p e$$

continues to produce the required PWM command.

> The steady-state error is what allows a pure P controller to keep producing a nonzero control effort.

13. PWM Saturation

The Arduino PWM output is limited to

$$0 \leq u \leq 255.$$

Suppose

$$r = 150 \text{ RPM}, \qquad y = 0, \qquad K_p = 5.$$

Initially,

$$e = 150 \text{ RPM}.$$

The ideal controller asks for

$$u = 5(150) = 750.$$

But the Arduino cannot output PWM 750.

Instead,

$$\boxed{u = \min(750, 255) = 255.}$$

This is called **saturation**.

> The mathematical controller can request any value, but the real actuator has physical limits. Saturation is one reason the real response will differ from the ideal closed-loop equation.

14. Practical P-Control Algorithm

For the Week 3 motor lab, the basic controller will look like:

```
// desired speed
float referenceRPM = 150.0;

// feedback error
float error = referenceRPM - measuredRPM;

// proportional controller
float control = Kp * error;

// PWM limit
int pwm = constrain((int)control, 0, 255);

// motor command
analogWrite(ENA, pwm);
```

Every sample, the controller updates the command using the newest speed measurement.

15. Check Your Understanding

Suppose

$$r = 120 \text{ RPM}, \qquad y = 90 \text{ RPM}, \qquad K_p = 3.$$

Answer:

1. What is the error?
2. What PWM command does the P controller request?
3. Is the requested PWM saturated?
4. If the measured speed increases to 110 RPM, what is the new controller output?

16. Practice with the Closed-Loop Model

A motor has

$$G(s) = \frac{0.8}{0.50s + 1}$$

and proportional gain

$$K_p = 4.$$

Find:

1. KK_p;
2. the closed-loop transfer function $Y(s)/R(s)$;
3. the closed-loop time constant;
4. the steady-state speed for $r = 100$ RPM;
5. the steady-state error.

Summary

- Feedback compares desired and measured speed.

- The error is
$$e = r - y.$$
- A proportional controller uses
$$u = K_p e.$$
- The closed-loop transfer function is
$$\frac{Y}{R} = \frac{K_p G}{1 + K_p G}.$$
- For $G(s) = K/(\tau s + 1)$,
$$\frac{Y}{R} = \frac{K K_p}{\tau s + 1 + K K_p}.$$
- Increasing K_p ideally reduces steady-state error and speeds up the response.
- Real hardware is limited by PWM saturation.